# Phase 2 — Persistence Layer

db.py · state.py · migrations.py · timeutil · defaults · paths · ~/trading-bot/data/

● phase 2 done

## sqlite — persistent · historical · analytically queryable

### persistence/db.py — Database

● 59 tests ✓

PRAGMA journal_mode = WAL · foreign_keys = ON · busy_timeout = 30000
synchronous = NORMAL · temp_store = MEMORY · cache_size = -64000 (64 MB)

tables — 7 total · phase 2 creates all · phase N writes data

- ● **bot_runs** run_uuid · phase · pid · started_at · ended_at · all phases
- ● **trade_outcomes** ← Phase 5
  deal_id · deal_reference* · epic · direction · size · open/close_level
- ● **reward_pts** ← Phase 7
  outcome_score_delta · score_after · win_loss · V2: process_score_delta
- ● **trade_lessons** ← Phase 7
  lesson_text · relevance_score · used_in_research · V2: embedding_json
- ● **ig_config_state** key TEXT PRIMARY KEY · value JSON · seeded by init
- ○ **market_regime_snapshots** volatility_pct · spread_pct · ★V2 Ph9
- ○ **decision_context** bull_summary · bear_summary · judge_reasoning · ★V2 Ph9

FK: trade_outcomes.run_id → bot_runs ON DELETE SET NULL
reward_pts · trade_lessons · market_regime_snapshots · decision_context
idx: deal_id · deal_reference* · session_date · epic · status · run_id

† session_date: Europe/Berlin local date — DAX trading day, not UTC calenda
* deal_reference: indexed for O(1) reconcile_positions lookup on bot restar

key methods

insert_trade() · get_trade_by_deal_id() · update_trade_close() · get_recent
record_trade_complete(trade, reward?, lesson?, regime?, context?) ← atomic txn
get_unreconciled_deal_references() ← restart-safe reconcile bridge
get_current_score() · get_risk_level() → KONSERVATIV / AGGRESSIV (threshold

### persistence/migrations.py · migrations/001_initial.sql

run_migrations(conn, dir) · _schema_version table · BEGIN IMMEDIATE · idemp
001_initial.sql: all 7 tables + indexes · next: 002_v2_extensions.sql (Phas

## json state — operational · ephemeral · millisecond access

### persistence/state.py — StateManager

atomic_write_json(path, data) → write tmp · fsync · os.replace() (POSIX atomic
TTL checks via parse_utc_iso() · state_dir constructor override for isolated te

### ig_state.json — account snapshot · updated on every get_account() call

{ balance, available, profit_loss, currency, session_active,
open_positions: [ { deal_id, epic, direction, size, open_level, P&L } ], last_u
TTL: fresh if last_updated < stale → bot re-fetches before any gate check

### ig_config.json — static config · no TTL · user-edited · loaded once at bot start

{ trading_window: { start_cet, end_cet }, risk: { max_risk_pct, position_cap_gb
filters: { max_spread_pct: 0.5, min_confidence: 55 }, research: { candidates_tt
veto_trigger_count

### turbo_candidates.json — research output · written by Phase 4 · consumed by Phase 5 G

{ generated_at, ttl_minutes: 30, expires_at (ISO 8601 UTC), session_date,
candidates: [ { epic, direction, confidence, reasoning, spread_pct_at_research
TTL: 30 min via explicit expires_at field
load_candidates() → [] on expiry · candidates_are_fresh() · clear_candidates()
save_candidates(candidates, ttl_minutes=30, session_date=...) — always writes f

### persistence/paths.py · persistence/timeutil.py

data_root() → ~/trading-bot/data/ · override: env TRADING_BOT_DATA_DIR · ensure_
session_date_from_utc_iso(s) → Europe/Berlin date · utc_iso_now() · BERLIN_TZ
berlin_now_decimal_hours() · parse_utc_iso() · zoneinfo stdlib (tzdata on Windo

### persistence/defaults.py — ig_config_state seed values

bot_score → 50.0 (neutral start · earn AGGRESSIV by pushing score above 50)
risk_level → "KONSERVATIV" · score ≤ 50 = KONSERVATIV · score > 50 = AGGRESSIV
daily_pnl_today → 0.0 · consec_losses → 0 · last_trade_id → null

### design decisions locked in Phase 2

isolation → persistence/ never imports broker_wrapper · data flows as plain dict
connection → one persistent sqlite3 conn per Database instance · context manager
WAL → Phase 5 (write) + Phase 8 (read) coexist without blocking
time lib → time-machine (not freezegun) — 100× faster · C-level mock
data dir → ~/trading-bot/data/ · gitignored · outside all phase dirs
force_trigger → NOT in defaults — anti-pattern per CLAUDE.md · no bypass lever

## Integration Contracts — Phase 1 ← → Phase 2 ← → Phase 5+

Phase 1 → 2 Envelope.data (dict) → insert_trade() · close_data dict → update_trade_close() · broker field already "ig"
Phase 5 → record_trade_complete(trade, reward?, lesson?, regime?, context?) ← all-or-nothing BEGIN IMMEDIATE transaction
Reconcile refs = db.get_unreconciled_deal_references() → broker.reconcile_positions(expected_references=refs)
Phase 7 → insert_reward(trade_id, reward) · insert_lesson(trade_id, lesson) · mark_lesson_used(lesson_id)

done (Ph1/Ph2) · Phase 5 write · migrations · Phase 7 write · ★V2 Phase 9 · JSON state

59 unit tests green · live_test.py verified · 2026-05-22